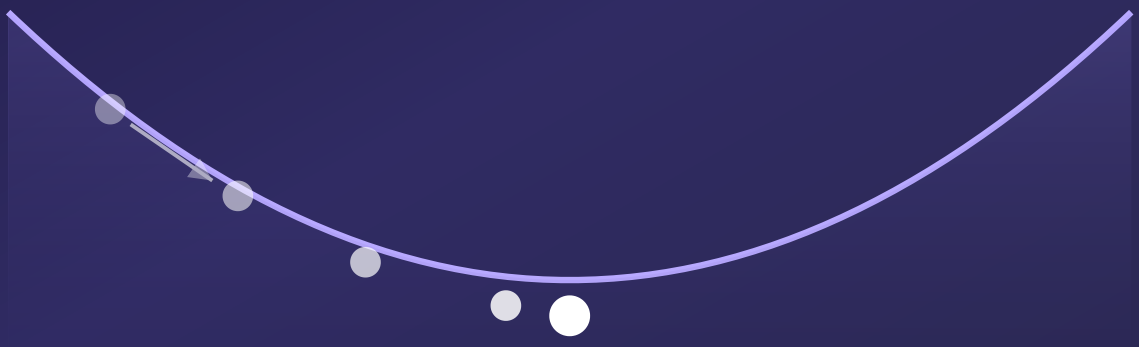
INTRODUCTORY COURSE · MACHINE LEARNING

Gradient Descent

How algorithms learn to find the best solution



minimum of the cost function

An accessible introduction, from intuition to math and code ·
10 pages · Beginner – Intermediate level

TABLE OF CONTENTS

What you will learn

> *This course takes you step by step from a simple intuition all the way to the real implementation of one of the most important algorithms in artificial intelligence.*

HOW TO USE THIS COURSE

Each chapter builds on the previous one. Don't skip the analogies — they connect the math to something concrete. Formulas are explained in plain language right after they appear.

CHAPTER 1

The intuition: walking down a mountain

> *Imagine you're on a mountain, in thick fog, and you want to reach the valley. You can't see anything around you. What do you do?*

You feel the slope around you with your foot and take a step in the direction where the ground descends most steeply. Then you stop, feel the slope again, and take another step. You repeat until the ground becomes flat — you've reached the valley. ***This is exactly what gradient descent does.***

In machine learning, the "mountain" is a **cost function**: a mathematical surface where the height represents how large the errors of our model are. High peaks mean bad predictions; the valley means good predictions. The goal of the algorithm is to find the lowest point possible.

Why do we need this algorithm?

A machine learning model has *parameters* — internal numbers it can adjust. A simple model may have a few parameters; a modern neural network can have billions. For each combination of parameters, the model produces a certain level of error.

The question is: **which parameter values produce the smallest error?** We can't try all combinations — there would be astronomically many. We need a smart method that guides us directly toward a good solution. Gradient descent is that method.

THE CORE IDEA

Gradient descent is an **optimization** algorithm: it starts from a random solution and improves it gradually, step by step, always moving "downhill" on the error surface, until it reaches (almost) its minimum.

Three ingredients

To climb down the mountain we need three things, which we'll study one by one in the following chapters:

- **A height map** — the cost function, which tells us how bad the current solution is (Chapter 2).
- **A compass** — the gradient, which shows us in which direction the ground descends most steeply (Chapter 3).
- **A step size** — the learning rate, which decides how far we move at each step (Chapter 5).

CHAPTER 2

The cost function

> *Before we can descend, we need to know what "lower" means. The cost function gives us this measure.*

A **cost function** (also called a loss function) takes the model's parameters and returns a single number: how far the model's predictions are from the actual values. The smaller the number, the better the model.

A concrete example: linear regression

Let's say we want to predict the price of a house based on its area. Our model is a straight line: $\hat{y} = w \cdot x + b$, where x is the area, $\hat{y}$ is the predicted price, and w and b are the parameters we want to find.

For each house in our data we compare the predicted price $\hat{y}$ with the actual price y. The difference $\hat{y} - y$ is the error for that house. A very common measure is the **mean squared error**:

COST FUNCTION — MEAN SQUARED ERROR

$$J(w, b) = \frac{1}{n} \sum_{i=1}^{n} \left(\hat{y}_i - y_i\right)^2$$

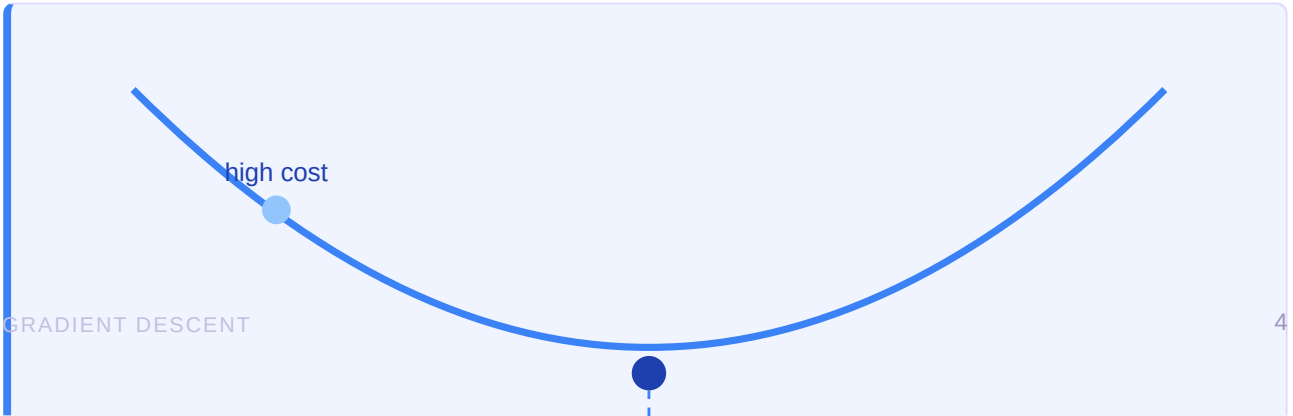
Let's read it in words. For each of the n examples: we take the difference between the prediction and the actual value, square it (so it's always positive and so that large errors count disproportionately more), then average all these squares. The result J is a single number that summarizes how well our line fits the data.

WHY DO WE SQUARE IT?

Two reasons. First: an error of -5 and one of $+5$ are equally bad, and squaring treats them identically (both become 25). Second: squaring heavily penalizes large errors, pushing the model to avoid them. Technical bonus — the resulting function is smooth and differentiable everywhere, exactly what we need for the next step.

The cost surface

If we plot the value of J for every possible combination of w and b, we get a bowl-shaped surface (a paraboloid). This is the "mountain" from our analogy. The lowest point of the bowl corresponds to the best line — the pair (w, b) we are looking for.



CHAPTER 3

The gradient: the algorithm's compass

> *The gradient is the concept that gives the algorithm its name. Good news: behind it lies an idea from high school — slope.*

The derivative: slope at a point

For a function of a single variable, the **derivative** tells us how fast the function increases or decreases at a point. We write it $\frac{dJ}{dw}$. If the derivative is positive, the function rises toward the right; if it's negative, the function falls toward the right.

Think of the derivative as the steepness of the ground right under your feet. A steep slope means a large derivative; flat ground means a derivative close to zero. And in the valley — the bottom of the bowl — the ground is perfectly flat, so the derivative is exactly zero.

The gradient: slope in many directions

Real models have many parameters, not just one. The **gradient** is simply the collection of all partial derivatives — one for each parameter. We denote it with the nabla symbol, ∇J:

THE GRADIENT — THE VECTOR OF ALL SLOPES

$$\nabla J = \left(\frac{\partial J}{\partial w_1}, \frac{\partial J}{\partial w_2}, \ldots, \frac{\partial J}{\partial w_m} \right)$$

Each component $\frac{\partial J}{\partial w_k}$ answers the question: "if I change only parameter w_k, keeping the rest fixed, how much does the cost change?" Together, these components form a vector that points in the direction of **steepest increase** of the cost.

THE KEY PROPERTY OF THE GRADIENT

The gradient ∇J always points in the direction in which the function **increases** fastest. But we want to descend! So we move in the **opposite** direction of the gradient: $-\nabla J$. This is the meaning of the word "descent" in the algorithm's name.

The gradient for linear regression

For the cost function from Chapter 2, the partial derivatives with respect to the two parameters are computed as follows:

PARTIAL DERIVATIVES OF THE MSE COST

$$\frac{\partial J}{\partial w} = \frac{2}{n} \sum_{i=1}^{n} (\hat{y}_i - y_i) x_i$$

$$\frac{\partial J}{\partial b} = \frac{2}{n} \sum_{i=1}^{n} (\hat{y}_i - y_i)$$

CHAPTER 4

The update rule

> *Now we have all the pieces. We combine them into a single formula — the heart of the gradient descent algorithm.*

The algorithm repeats the same simple step, over and over: it takes the current parameters, computes the gradient, and adjusts the parameters in the opposite direction of the gradient. In mathematical form:

THE GRADIENT DESCENT UPDATE RULE

$$\theta_{\text{new}} = \theta_{\text{old}} - \alpha \cdot \nabla J(\theta_{\text{old}})$$

Let's dissect it piece by piece, because this formula sums up the entire course:

- θ (theta) represents the model's parameters — all the numbers we adjust (for example w and b).
- $\nabla J(\theta)$ is the gradient — the compass that points uphill. The minus sign in front of it makes us go down, not up.
- α (alpha) is the **learning rate** — a small number that controls how big the step is. We study it in detail in the next chapter.
- The $=$ sign here means "assignment": the new value replaces the old one, and we repeat.

In mountain terms: θ_{old} is your current position, ∇J is the uphill direction you feel with your foot, the minus sign turns you toward the valley, and α decides how big a step you take. After the step, you're at θ_{new}, from where you start all over.

The complete algorithm, in five steps

1. **Initialize** the parameters with random values (some starting point on the mountain).
2. **Compute the gradient** ∇J at the current point — the local slope.
3. **Update** the parameters with the rule above — take a step downhill.
4. **Repeat** steps 2 and 3 many times.
5. **Stop** when the gradient becomes nearly zero (the ground is flat) or after a fixed number of iterations.

CONVERGENCE

As we get closer to the minimum, the slope decreases, so the gradient decreases too. This means the steps automatically become smaller near the target — the algorithm naturally "slows down" as it approaches the solution. We say the algorithm **converges** when the updates become negligibly small.

One question remains open: how big should α be? The answer is surprisingly important and deserves a whole chapter.

CHAPTER 5

The learning rate

> *The learning rate α seems like a minor detail, but it's probably the most important setting in the whole algorithm. Step size decides whether you succeed or fail.*

The learning rate controls how far we move at each iteration. Choosing it is a balancing act between speed and safety.

TOO SMALL

The steps are tiny. The algorithm eventually reaches the minimum, but it needs an enormous number of iterations. Training becomes slow and costly.

TOO LARGE

The steps are giant. The algorithm jumps over the minimum from one side to the other, oscillates, and may even *diverge* — the cost grows to infinity instead of decreasing.

The same problem, three different learning rates

How do we choose a good value?

There is no universal magic value — it depends on the problem. In practice, one often starts from values such as **0.1**, **0.01** or **0.001** and watches how the cost evolves. A few useful principles:

- If the cost **decreases smoothly and steadily**, the rate is probably good.
- If the cost **oscillates or explodes**, the rate is too large — decrease it.
- If the cost decreases **extremely slowly**, the rate is too small — increase it.

ADVANCED TECHNIQUE: ADAPTIVE RATES

Modern algorithms such as **Adam**, **RMSProp** or **AdaGrad** automatically adjust the learning rate during training — they start with larger steps and shrink them along the way. That's why they are used by default in most deep learning libraries. At their core, though, they all start from the same gradient descent idea.

CHAPTER 6

Variants: Batch, Stochastic, Mini-batch

> *So far we've assumed we use all the data at every step. When we have millions of examples, that becomes impractical. Here are the three variants that solve the problem.*

The underlying question

To compute the exact gradient, we have to go through all the training examples. But if we have 10 million examples, each step becomes very expensive. The three variants differ in **how many examples they use** to estimate the gradient at each step.

Variant	Examples per step	Characteristics
Batch (full)	All, at every step	Exact gradient and a smooth trajectory, but very slow on large datasets. Requires a lot of memory.
Stochastic (SGD)	A single example, chosen at random	Very fast and uses little memory. The trajectory is noisy, "zig-zaggy", but this noise can help escape weak local minima.
Mini-batch	A small group (e.g. 32, 64, 128)	The ideal compromise: fast enough, stable enough. Takes advantage of hardware optimizations. It is the standard in practice.

Trajectories toward the minimum: smooth, noisy, and balanced

A few useful terms

When working with these variants you'll come across two frequently used terms:

- **Epoch**: one complete pass through the entire training dataset. Training usually takes many epochs.
- **Batch size**: how many examples are grouped into a single update step. A common choice is between 32 and 256.

IN PRACTICE

Almost all modern machine learning systems use **mini-batch gradient descent**, often combined with an adaptive optimizer like Adam. It's the sweet spot between speed, stability and hardware efficiency.

CHAPTER 7

Implementation in Python

> *Theory comes alive in code. Here is the complete gradient descent for linear regression, in a few lines of pure Python — no magic libraries.*

The code below finds the best parameters w and b for a linear relationship between x and y. Watch how each line corresponds directly to the formulas from the previous chapters.

```python
import numpy as np

# Training data: x = input, y = actual output
x = np.array([1, 2, 3, 4, 5])
y = np.array([3, 5, 7, 9, 11])  # true relation: y = 2x + 1

# 1. Initialize the parameters with arbitrary values
w, b = 0.0, 0.0
alpha = 0.01      # learning rate
epochs = 1000     # how many iterations we run
n = len(x)

for i in range(epochs):
    # 2. Current prediction: y_hat = w*x + b
    y_pred = w * x + b

    # The error: difference between prediction and actual
    error = y_pred - y

    # 3. Compute the gradient (partial derivatives)
    grad_w = (2/n) * np.sum(error * x)
    grad_b = (2/n) * np.sum(error)

    # 4. Update the parameters: theta = theta - alpha*grad
    w = w - alpha * grad_w
    b = b - alpha * grad_b

print(f"w = {w:.3f}, b = {b:.3f}")
# Result: w ~ 2.000, b ~ 1.000  ->  it learned y = 2x + 1
```

What happens, line by line

- We start with $w = 0$ and $b = 0$ — a completely wrong model at the beginning.
- At each epoch we compute the predictions and errors for all examples simultaneously (thanks to NumPy's vector operations).
- We compute the gradient using exactly the formulas from Chapter 3.
- We apply the update rule from Chapter 4, moving w and b a little closer to the solution.
- After 1000 repetitions, the parameters converge toward the true values: $w = 2$, $b = 1$.

FROM SCRATCH TO FRAMEWORKS

CHAPTER 8

Pitfalls, tips and recap

> *You've walked the entire path, from intuition to code. Here's what to avoid, what to remember, and where to go next.*

Common pitfalls

- **Local minima.** On complex surfaces (neural networks), there can be several "valleys". The algorithm might stop in one that isn't the deepest. The noise in SGD and modern optimizers help mitigate the problem.
- **Unnormalized data.** If parameters have very different scales, the cost surface becomes elongated, making descent inefficient. The fix: normalize the input data before training.
- **Wrong learning rate.** The most common cause of failure. If training "explodes" or stalls, the first thing to adjust is α.
- **Too many or too few iterations.** Too few: the model doesn't get to learn. Too many: wasted time after it has already converged.

RECAP IN THREE SENTENCES

Gradient descent looks for the parameters that minimize a cost function, repeatedly moving "downhill" on the error surface.

At each step, the gradient points uphill, and we move in the opposite direction, by an amount given by the learning rate.

The central formula $\theta_{\text{new}} = \theta_{\text{old}} - \alpha \nabla J$ sums up the whole algorithm, from linear regression to gigantic neural networks.

Why it matters

Gradient descent is the engine that trains almost every modern machine learning model — from simple regressions to language models with billions of parameters. Understanding it gives you the key to grasping how artificial intelligence actually "learns": not by magic, but through small, repeated steps of improvement.

Where to go next

- **Backpropagation** — the algorithm that efficiently computes gradients in neural networks with many layers.
- **Advanced optimizers** — study Adam, RMSProp and momentum to see how convergence is accelerated.
- **Regularization** — techniques (L1, L2, dropout) that modify the cost function to avoid overfitting.
- **Practice** — implement gradient descent for logistic regression, then for a small neural network.

CONGRATULATIONS

You've understood one of the most important concepts in artificial intelligence. The next step is to put it into practice on real data — that's where intuition turns into mastery.